

Chapter 14 — Monotonic Stack

0. What You Will Learn in This Chapter

Monotonic Stack is one of the most important advanced stack patterns.

A normal stack stores recent unresolved items.

A **monotonic stack** stores recent unresolved items while maintaining a useful order:

- increasing order
- decreasing order
- non-increasing order
- non-decreasing order

This pattern is used when a problem asks for:

- next greater element
- next smaller element
- previous greater element
- previous smaller element
- nearest greater/smaller value
- span/count based on nearest boundary
- rectangle area
- contribution of each element as minimum/maximum
- removing digits greedily
- building lexicographically smallest result
- stock span
- daily temperatures
- histogram problems

By the end of this chapter, you should understand:

- What a monotonic stack is
- Why monotonic stack exists
- How it improves brute force from $O(n^2)$ to $O(n)$
- When to use increasing vs decreasing stack
- How to solve next greater / next smaller problems
- How to solve previous greater / previous smaller problems
- How to handle duplicates
- How to use indices instead of values
- How to solve circular next greater problems
- How to solve histogram rectangle problems
- How to solve contribution-counting problems
- How monotonic stack connects to greedy problems
- How to explain monotonic stack in interviews

Core problems covered in this chapter:

1. Next Greater Element I
2. Next Greater Element II
3. Daily Temperatures
4. Online Stock Span
5. Final Prices With a Special Discount
6. Remove K Digits
7. Remove Duplicate Letters
8. Largest Rectangle in Histogram
9. Maximal Rectangle
10. Sum of Subarray Minimums

11. Sum of Subarray Ranges
 12. Trapping Rain Water, stack version
 13. Asteroid Collision, stack review
 14. Pattern 132
 15. Car Fleet, monotonic-stack-style reasoning
-

1. What Is a Monotonic Stack?

A monotonic stack is a stack that keeps elements in a sorted-like order. There are two common types.

1.1 Monotonic Increasing Stack

The stack values increase from bottom to top.

Example:

bottom -> 1, 3, 5, 8 <- top

Usually used to find:

- next smaller element
- previous smaller element
- boundaries where current element is smaller

1.2 Monotonic Decreasing Stack

The stack values decrease from bottom to top.

Example:

bottom -> 9, 7, 4, 2 <- top

Usually used to find:

- next greater element
 - previous greater element
 - boundaries where current element is greater
-

2. Why Monotonic Stack Exists

Many problems ask:

For each index, find the next greater element to the right.

Brute force:

```
for each i:
  scan j from i + 1 to n - 1
  find first nums[j] > nums[i]
```

This is $O(n^2)$.

Monotonic stack improves this because each element is:

- pushed once
- popped at most once

So total work is $O(n)$.

3. Core Intuition

The stack stores elements that are still waiting for an answer.

Example:

```
nums = [2, 1, 2, 4, 3]
```

For next greater element:

- 2 waits for something greater.
- 1 waits for something greater.
- When another 2 arrives, it resolves 1.
- When 4 arrives, it resolves both 2s.

The current element can resolve previous unresolved elements.

That is the heart of monotonic stack:

Current element may be the answer for earlier elements.

4. Core Need / Why Use Monotonic Stack Over Normal Stack?

A normal stack only gives recent item access.

A monotonic stack gives recent item access **plus ordering**.

This ordering helps remove useless candidates.

Example:

For next greater element, suppose stack contains:

```
[5, 3, 2]
```

Now current value is 4.

4 is greater than 2 and 3.

So:

- next greater of 2 is 4
- next greater of 3 is 4

But 5 remains unresolved.

So we pop smaller values until stack top is greater than current.

5. When to Use Monotonic Stack

Use monotonic stack when the problem asks for one of these.

5.1 Next Greater Element

Clues:

- next greater
- warmer temperature
- first greater value to the right
- days until a greater value

Examples:

- Next Greater Element I
- Next Greater Element II
- Daily Temperatures

5.2 Next Smaller Element

Clues:

- next smaller
- first discount less than or equal

- first lower value to the right

Examples:

- Final Prices With a Special Discount
- Remove K Digits
- Histogram boundaries

5.3 Previous Greater / Previous Smaller

Clues:

- stock span
- nearest greater to left
- nearest smaller to left
- count consecutive previous values satisfying condition

Examples:

- Online Stock Span
- Largest Rectangle in Histogram

5.4 Boundary Problems

Clues:

- largest rectangle
- subarray where element is minimum
- subarray where element is maximum
- contribution as min/max

Examples:

- Largest Rectangle in Histogram
- Sum of Subarray Minimums
- Sum of Subarray Ranges

5.5 Greedy Removal

Clues:

- remove k digits
- smallest lexicographic result
- remove characters while maintaining order
- current smaller character should remove previous larger character

Examples:

- Remove K Digits
 - Remove Duplicate Letters
 - Smallest Subsequence of Distinct Characters
-

6. When NOT to Use Monotonic Stack

6.1 Need Arbitrary Lookup

Use hash map or set.

Monotonic stack only helps with nearest greater/smaller relationships.

6.2 Need Sorted Output

Use sorting.

Monotonic stack does not sort the array.

6.3 Need Sliding Window Maximum

Use monotonic deque, not stack.

That comes in the next chapter.

6.4 Need Global Maximum or Minimum Only

Use basic traversal.

Do not use monotonic stack if the problem only asks:

maximum element in array

6.5 Need All Pairs

Monotonic stack usually finds nearest boundary, not all combinations.

7. Mental Models

7.1 Stack of Unresolved Elements

For next greater:

The stack contains elements still waiting for a greater element.

When current value is greater than stack top, current resolves stack top.

7.2 Stronger Element Removes Weaker Element

For next greater:

Current larger value removes smaller previous values.

For next smaller:

Current smaller value removes larger previous values.

7.3 Boundary Finder

For histogram:

Each bar wants to know:

- how far left can I extend?
- how far right can I extend?

Monotonic stack finds the first smaller boundary on both sides.

7.4 Contribution Counter

For subarray minimums:

For each element, count:

- how many subarrays use it as minimum

This requires knowing previous smaller and next smaller boundaries.

8. Pattern Recognition Checklist

Ask:

16. Does the problem ask for next greater or next smaller?
17. Does it ask for nearest greater/smaller value?
18. Does current element resolve previous elements?
19. Do I need first greater/smaller on the right?
20. Do I need previous greater/smaller on the left?
21. Is brute force scanning right for each index $O(n^2)$?
22. Can each element be pushed and popped once?
23. Does the problem involve histogram rectangles?
24. Does each element contribute as min/max to many subarrays?
25. Is there a greedy removal of previous larger/smaller elements?

If yes, monotonic stack is likely useful.

9. C++ Tools Needed

9.1 `stack<int>`

```
stack<int> st;
```

Good for simple cases.

9.2 `vector<int>` as Stack

Often preferred because it allows easier access and iteration.

```
vector<int> st;
```

```
st.push_back(i);
```

```
st.back();
```

```
st.pop_back();
```

```
st.empty();
```

9.3 Store Indices, Not Values

Most monotonic stack problems store indices.

Why?

Because index lets you access:

```
nums[index]
```

and calculate distances:

```
i - index
```

Example:

Daily Temperatures needs number of days:

```
answer[prevIndex] = i - prevIndex;
```

So store indices.

10. Core Templates

Template 1 — Next Greater Element to Right

Use decreasing stack.

```
vector<int> nextGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, -1);
    vector<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[i] > nums[st.back()]) {
            int index = st.back();
            st.pop_back();
            answer[index] = i;
        }
        st.push_back(i);
    }
}
```

```
        answer[index] = nums[i];
    }

    st.push_back(i);
}

return answer;
}
```

Template 2 — Next Smaller Element to Right

Use increasing stack.

```
vector<int> nextSmaller(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, -1);
    vector<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[i] < nums[st.back()]) {
            int index = st.back();
            st.pop_back();

            answer[index] = nums[i];
        }

        st.push_back(i);
    }

    return answer;
}
```

Template 3 — Previous Greater Element

Scan left to right.

Maintain decreasing stack.

```
vector<int> previousGreater(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, -1);
    vector<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[st.back()] <= nums[i]) {
            st.pop_back();
        }
    }
}
```

```

    }

    if (!st.empty()) {
        answer[i] = nums[st.back()];
    }

    st.push_back(i);
}

return answer;
}

```

Template 4 — Previous Smaller Element

Maintain increasing stack.

```

vector<int> previousSmaller(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, -1);
    vector<int> st;

    for (int i = 0; i < n; i++) {
        while (!st.empty() && nums[st.back()] >= nums[i]) {
            st.pop_back();
        }

        if (!st.empty()) {
            answer[i] = nums[st.back()];
        }

        st.push_back(i);
    }

    return answer;
}

```

Template 5 — Circular Next Greater

Loop twice.

```

vector<int> nextGreaterCircular(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, -1);
    vector<int> st;

```

```

for (int i = 0; i < 2 * n; i++) {
    int index = i % n;

    while (!st.empty() && nums[index] > nums[st.back()]) {
        answer[st.back()] = nums[index];
        st.pop_back();
    }

    if (i < n) {
        st.push_back(index);
    }
}

return answer;
}

```

Template 6 — Histogram With Sentinel

```

int largestRectangleArea(vector<int>& heights) {
    heights.push_back(0);

    vector<int> st;
    int best = 0;

    for (int i = 0; i < heights.size(); i++) {
        while (!st.empty() && heights[i] < heights[st.back()]) {
            int h = heights[st.back()];
            st.pop_back();

            int left = st.empty() ? -1 : st.back();
            int width = i - left - 1;

            best = max(best, h * width);
        }

        st.push_back(i);
    }

    heights.pop_back();

    return best;
}

```

11. Major Variations of Monotonic Stack

Next greater	decreasing stack	Daily Temperatures
Next smaller	increasing stack	Final Prices
Previous greater	decreasing stack	Stock Span
Previous smaller	increasing stack	Histogram
Circular next greater	decreasing stack + 2 loops	Next Greater II
Histogram rectangle	increasing stack	Largest Rectangle
Contribution as minimum	prev/next smaller	Sum of Subarray Minimums
Greedy remove bigger previous	increasing stack-like	Remove K Digits
Lexicographic stack	increasing with seen set	Remove Duplicate Letters
Water trapping stack	decreasing boundary stack	Trapping Rain Water

12. Full Problem Walkthrough 1 — LeetCode 496: Next Greater Element I

12.1 Problem Summary

You are given two arrays:

```
nums1
nums2
```

nums1 is a subset of nums2.

For each element in nums1, find its next greater element in nums2.

Example:

```
nums1 = [4,1,2]
nums2 = [1,3,4,2]
```

Output:

```
[-1,3,-1]
```

12.2 How to Identify This Pattern

The problem directly asks:

next greater element

This is a monotonic decreasing stack problem.

12.3 Brute Force Idea

For each number in nums1, find it in nums2, then scan right until a greater element appears.

Time:

$O(n * m)$

Too slow.

12.4 Optimized Intuition

Precompute next greater for every number in nums2.

Use a decreasing stack.

When current number is greater than stack top, it is the next greater for stack top.

12.5 C++ Code

```
class Solution {
public:
    vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
        unordered_map<int, int> nextGreater;
        vector<int> st;

        for (int x : nums2) {
            while (!st.empty() && x > st.back()) {
                nextGreater[st.back()] = x;
                st.pop_back();
            }

            st.push_back(x);
        }

        vector<int> answer;
```

```

    for (int x : nums1) {
        if (nextGreater.count(x)) {
            answer.push_back(nextGreater[x]);
        } else {
            answer.push_back(-1);
        }
    }

    return answer;
}
};

```

12.6 Dry Run

nums2 = [1,3,4,2]

1	push	1	
3	3 resolves 1	3	1 -> 3
4	4 resolves 3	4	1 -> 3, 3 -> 4
2	push	4, 2	

Unresolved values get -1.

12.7 Complexity

Time:

$O(n + m)$

Space:

$O(m)$

12.8 Common Mistakes

- Using nested loops
- Forgetting unresolved values should be -1
- Using stack but not hash map for lookup from nums1

13. Full Problem Walkthrough 2 — LeetCode 503: Next Greater Element II

13.1 Problem Summary

Given a circular array, return next greater element for every index.

Example:

nums = [1,2,1]

Output:

[2,-1,2]

For last 1, next greater is 2 because array is circular.

13.2 How to Identify This Pattern

Problem says:

- next greater
- circular array

Use monotonic stack and simulate two passes.

13.3 C++ Code

```
class Solution {
public:
    vector<int> nextGreaterElements(vector<int>& nums) {
        int n = nums.size();

        vector<int> answer(n, -1);
        vector<int> st;

        for (int i = 0; i < 2 * n; i++) {
            int index = i % n;

            while (!st.empty() && nums[index] > nums[st.back()]) {
                answer[st.back()] = nums[index];
                st.pop_back();
            }

            if (i < n) {
                st.push_back(index);
            }
        }

        return answer;
    }
};
```

13.4 Why Push Only During First Pass?

We only need each index in stack once.

Second pass is only to resolve elements using circular continuation.

13.5 Dry Run

nums = [1,2,1]

First pass:

- index 0 waits
- index 1 resolves index 0
- index 2 waits

Second pass:

- index 1 resolves index 2

Answer:

[2,-1,2]

13.6 Complexity

Time:

$O(n)$

Even though loop runs $2n$, that is still $O(n)$.

Space:

$O(n)$

14. Full Problem Walkthrough 3 — LeetCode 739: Daily Temperatures

14.1 Problem Summary

Given daily temperatures, return for each day how many days you must wait until a warmer temperature.

If none, return 0.

Example:

temperatures = [73,74,75,71,69,72,76,73]

Output:

[1,1,4,2,1,1,0,0]

14.2 How to Identify This Pattern

Problem asks:

next warmer day

That means next greater element by temperature.

Need distance, so store indices.

14.3 C++ Code

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n = temperatures.size();

        vector<int> answer(n, 0);
        vector<int> st;

        for (int i = 0; i < n; i++) {
            while (!st.empty() && temperatures[i] > temperatures[st.back()]) {
                int prev = st.back();
                st.pop_back();

                answer[prev] = i - prev;
            }

            st.push_back(i);
        }

        return answer;
    }
};
```

14.4 Dry Run

Input:

[73,74,75,71,69,72,76,73]

0	73	push 0
1	74	resolves 0, answer[0] = 1
2	75	resolves 1, answer[1] = 1
3	71	push 3
4	69	push 4
5	72	resolves 4 and 3
6	76	resolves 5 and 2
7	73	push 7

14.5 Complexity

Time:

$O(n)$

Each index is pushed once and popped once.

Space:

$O(n)$

14.6 Common Mistakes

- Storing values instead of indices
 - Returning warmer temperature instead of days
 - Using $\geq$ instead of $>$; equal temperature is not warmer
-

15. Full Problem Walkthrough 4 — LeetCode 901: Online Stock Span

15.1 Problem Summary

For each stock price, return span.

Span means number of consecutive days before today, including today, where price was less than or equal to today's price.

Example:

prices = [100,80,60,70,60,75,85]

Output:

[1,1,1,2,1,4,6]

15.2 How to Identify This Pattern

For each price, we need previous greater price boundary.

All previous prices less than or equal to current get absorbed.

Use decreasing stack of:

price, span

15.3 C++ Code

```
class StockSpanner {
private:
    vector<pair<int, int>> st;

public:
    StockSpanner() {
    }
}
```

```

int next(int price) {
    int span = 1;

    while (!st.empty() && st.back().first <= price) {
        span += st.back().second;
        st.pop_back();
    }

    st.push_back({price, span});

    return span;
}
};

```

15.4 Dry Run

100	push	1
80	push	1
60	push	1
70	absorbs 60	2
60	push	1
75	absorbs 60 and 70	4
85	absorbs 75 and 80	6

15.5 Complexity

Amortized time per next:

$O(1)$

Because each price is pushed once and popped once.

Space:

$O(n)$

16. Full Problem Walkthrough 5 — LeetCode 1475: Final Prices With a Special Discount

16.1 Problem Summary

For each price, subtract the first price to its right that is less than or equal to it.

Example:

```
prices = [8,4,6,2,3]
```

Output:

```
[4,2,4,2,3]
```

16.2 How to Identify This Pattern

For each element, need next smaller or equal element.

Use increasing stack.

16.3 C++ Code

```
class Solution {
public:
    vector<int> finalPrices(vector<int>& prices) {
        int n = prices.size();

        vector<int> answer = prices;
        vector<int> st;

        for (int i = 0; i < n; i++) {
            while (!st.empty() && prices[i] <= prices[st.back()]) {
                int index = st.back();
                st.pop_back();

                answer[index] = prices[index] - prices[i];
            }

            st.push_back(i);
        }

        return answer;
    }
};
```

16.4 Dry Run

prices = [8,4,6,2,3]

0	8	push
1	4	discount for 8 is 4
2	6	push
3	2	discount for 6 and 4 is 2
4	3	push

Output:

[4,2,4,2,3]

16.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

17. Full Problem Walkthrough 6 — LeetCode 402: Remove K Digits

17.1 Problem Summary

Given a numeric string num and integer k, remove exactly k digits so the remaining number is smallest possible.

Example:

num = "1432219"

k = 3

Output:

"1219"

17.2 How to Identify This Pattern

To make number smaller, remove a previous larger digit when a smaller digit appears after it.

This is monotonic increasing stack.

17.3 Greedy Intuition

If stack top is greater than current digit, removing stack top makes number smaller earlier.

Example:

14...

If current is 3, remove 4.

17.4 C++ Code

```
class Solution {
public:
    string removeKdigits(string num, int k) {
        string st;

        for (char c : num) {
            while (!st.empty() && k > 0 && st.back() > c) {
                st.pop_back();
                k--;
            }

            st.push_back(c);
        }

        while (k > 0 && !st.empty()) {
            st.pop_back();
            k--;
        }

        int index = 0;

        while (index < st.size() && st[index] == '0') {
            index++;
        }

        string result = st.substr(index);

        return result.empty() ? "0" : result;
    }
};
```

17.5 Dry Run

num = "1432219"

k = 3

1	push	1
4	push	14
3	remove 4	13
2	remove 3	12
2	push	122
1	remove 2	121
9	push	1219

Output:

1219

17.6 Complexity

Time:

$O(n)$

Space:

$O(n)$

17.7 Common Mistakes

- Forgetting to remove remaining digits after loop
- Not removing leading zeroes
- Returning empty string instead of "0"
- Using local smallest digit logic instead of stack

18. Full Problem Walkthrough 7 — LeetCode 316: Remove Duplicate Letters

18.1 Problem Summary

Given a string, remove duplicate letters so every letter appears once and result is smallest lexicographically.

Example:

$s = \text{"cbacdcbc"}$

Output:

"acdb"

18.2 How to Identify This Pattern

Need:

- each character once
- smallest lexicographic order
- preserve relative order
- remove previous larger character if it appears later again

This is greedy monotonic stack.

18.3 Key Conditions for Popping

We can pop stack top if:

26. stack top is greater than current character
 27. stack top appears again later
 28. current character is not already used
-

18.4 C++ Code

```
class Solution {
public:
    string removeDuplicateLetters(string s) {
        vector<int> last(26, 0);
        vector<bool> used(26, false);

        for (int i = 0; i < s.size(); i++) {
            last[s[i] - 'a'] = i;
        }

        string st;

        for (int i = 0; i < s.size(); i++) {
            char c = s[i];
            int id = c - 'a';

            if (used[id]) {
                continue;
            }

            while (!st.empty() &&
                   st.back() > c &&
                   last[st.back() - 'a'] > i) {
                used[st.back() - 'a'] = false;
                st.pop_back();
            }
        }
    }
};
```

```
        st.push_back(c);
        used[id] = true;
    }

    return st;
}
};
```

18.5 Dry Run Summary

Input:

cbacdcbc

- c pushed
- b removes c because c appears later
- a removes b because b appears later
- result eventually becomes acdb

18.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

Only 26 lowercase letters.

18.7 Similar Problems

- Smallest Subsequence of Distinct Characters
- Remove K Digits
- Create Maximum Number

19. Full Problem Walkthrough 8 — LeetCode 84: Largest Rectangle in Histogram

19.1 Problem Summary

Given bar heights, find largest rectangle area in histogram.

Example:

heights = [2,1,5,6,2,3]

Output:

10

Best rectangle uses heights 5 and 6 with height 5, width 2.

19.2 How to Identify This Pattern

For each bar, need to know:

- first smaller bar on left
- first smaller bar on right

That gives maximum width where this bar is the limiting height.

Use increasing monotonic stack.

19.3 Core Intuition

When current height is smaller than stack top, current becomes the right boundary for stack top.

After popping a height:

- right boundary is current index
 - left boundary is new stack top
 - width is right - left - 1
-

19.4 C++ Code

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        vector<int> st;
        int best = 0;

        heights.push_back(0);

        for (int i = 0; i < heights.size(); i++) {
            while (!st.empty() && heights[i] < heights[st.back()]) {
                int h = heights[st.back()];
                st.pop_back();

                int leftBoundary = st.empty() ? -1 : st.back();
                int width = i - leftBoundary - 1;

                best = max(best, h * width);
            }

            st.push_back(i);
        }

        heights.pop_back();

        return best;
    }
};
```

```
}  
};
```

19.5 Dry Run

```
heights = [2,1,5,6,2,3]
```

Key moment at height 2 index 4:

- Pop height 6: width 1, area 6
- Pop height 5: width 2, area 10

Best becomes 10.

19.6 Why Add Sentinel 0?

The sentinel forces all remaining bars to be processed.

Without it, bars left in stack may never be popped.

19.7 Complexity

Time:

```
O(n)
```

Each index is pushed once and popped once.

Space:

```
O(n)
```

19.8 Common Mistakes

- Forgetting sentinel
- Wrong width formula
- Using current index as left boundary
- Using values instead of indices
- Mishandling equal heights

20. Full Problem Walkthrough 9 — LeetCode 85: Maximal Rectangle

20.1 Problem Summary

Given a binary matrix of '0' and '1', find largest rectangle containing only 1s.

Example:

```
matrix =  
[  
  ["1","0","1","0","0"],  
  ["1","0","1","1","1"],  
  ["1","1","1","1","1"],  
  ["1","0","0","1","0"]  
]
```

Output:

6

20.2 How to Identify This Pattern

Each row can be treated as base of a histogram.

For every row:

- $height[j]$ = consecutive ones ending at this row in column j

Then solve Largest Rectangle in Histogram.

20.3 C++ Code

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        vector<int> st;
        int best = 0;

        heights.push_back(0);

        for (int i = 0; i < heights.size(); i++) {
            while (!st.empty() && heights[i] < heights[st.back()]) {
                int h = heights[st.back()];
                st.pop_back();

                int left = st.empty() ? -1 : st.back();
                int width = i - left - 1;

                best = max(best, h * width);
            }

            st.push_back(i);
        }

        heights.pop_back();

        return best;
    }

    int maximalRectangle(vector<vector<char>>& matrix) {
        if (matrix.empty()) {
            return 0;
        }
    }
```

```

int rows = matrix.size();
int cols = matrix[0].size();

vector<int> heights(cols, 0);
int best = 0;

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (matrix[r][c] == '1') {
            heights[c]++;
        } else {
            heights[c] = 0;
        }
    }

    best = max(best, largestRectangleArea(heights));
}

return best;
}
};

```

20.4 Complexity

Let rows = R and cols = C.

Time:

$O(R * C)$

Each row runs histogram in $O(C)$.

Space:

$O(C)$

20.5 Similar Problems

- Largest Rectangle in Histogram
- Maximal Square
- Count Square Submatrices

21. Full Problem Walkthrough 10 — LeetCode 907: Sum of Subarray Minimums

21.1 Problem Summary

Given array arr, return sum of minimum value of every subarray.

Example:

arr = [3,1,2,4]

Subarrays and minimums:

[3] -> 3
[3,1] -> 1
[3,1,2] -> 1
[3,1,2,4] -> 1
[1] -> 1
[1,2] -> 1
[1,2,4] -> 1
[2] -> 2
[2,4] -> 2
[4] -> 4

Sum:

17

21.2 How to Identify This Pattern

Problem asks sum over all subarrays.

Brute force is $O(n^2)$.

Instead, calculate contribution of each element as the minimum.

Need boundaries where element remains minimum.

Use monotonic stack.

21.3 Contribution Formula

For each index i:

leftCount = number of choices for subarray start
rightCount = number of choices for subarray end
contribution = arr[i] * leftCount * rightCount

Need:

- previous strictly smaller element
- next smaller or equal element

This duplicate handling prevents double counting.

21.4 C++ Code

```
class Solution {
public:
    int sumSubarrayMins(vector<int>& arr) {
        const int MOD = 1000000007;
        int n = arr.size();
```

```

vector<int> prevLess(n);
vector<int> nextLessEqual(n);

vector<int> st;

for (int i = 0; i < n; i++) {
    while (!st.empty() && arr[st.back()] >= arr[i]) {
        st.pop_back();
    }

    prevLess[i] = st.empty() ? -1 : st.back();
    st.push_back(i);
}

st.clear();

for (int i = n - 1; i >= 0; i--) {
    while (!st.empty() && arr[st.back()] > arr[i]) {
        st.pop_back();
    }

    nextLessEqual[i] = st.empty() ? n : st.back();
    st.push_back(i);
}

long long answer = 0;

for (int i = 0; i < n; i++) {
    long long left = i - prevLess[i];
    long long right = nextLessEqual[i] - i;

    answer = (answer + arr[i] * left * right) % MOD;
}

return answer;
}
};

```

21.5 Why One Side Strict and One Side Non-Strict?

Duplicates can cause double counting.

Using:

- previous strictly less
- next less or equal

assigns equal-value subarrays consistently to one side.

Another valid pairing is:

- previous less or equal
- next strictly less

But do not use both strict or both non-strict blindly.

21.6 Complexity

Time:

$O(n)$

Space:

$O(n)$

21.7 Common Mistakes

- Double-counting duplicates
 - Forgetting modulo
 - Trying $O(n^2)$
 - Wrong boundary direction
 - Using int for multiplication
-

22. Full Problem Walkthrough 11 — LeetCode 2104: Sum of Subarray Ranges

22.1 Problem Summary

The range of a subarray is:

maximum - minimum

Return sum of ranges of all subarrays.

Example:

nums = [1,2,3]

Subarray ranges:

[1] -> 0
[2] -> 0
[3] -> 0
[1,2] -> 1
[2,3] -> 1
[1,2,3] -> 2

Sum:

4

22.2 How to Identify This Pattern

Sum of ranges:

sum of subarray maximums - sum of subarray minimums

Use monotonic stack contribution twice.

22.3 C++ Code

```
class Solution {
public:
    long long subArrayRanges(vector<int>& nums) {
        int n = nums.size();

        long long sumMax = 0;
        long long sumMin = 0;

        vector<int> st;

        // Contribution as maximum
        for (int i = 0; i <= n; i++) {
            while (!st.empty() && (i == n || nums[st.back()] < nums[i])) {
                int mid = st.back();
                st.pop_back();

                int left = st.empty() ? -1 : st.back();
                int right = i;

                sumMax += 1LL * nums[mid] * (mid - left) * (right - mid);
            }

            st.push_back(i);
        }

        st.clear();

        // Contribution as minimum
        for (int i = 0; i <= n; i++) {
            while (!st.empty() && (i == n || nums[st.back()] > nums[i])) {
                int mid = st.back();
                st.pop_back();

                int left = st.empty() ? -1 : st.back();
                int right = i;
```

```
        sumMin += 1LL * nums[mid] * (mid - left) * (right - mid);
    }

    st.push_back(i);
}

return sumMax - sumMin;
}
};
```

22.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

22.5 Common Mistakes

- Calculating each subarray directly
- Not separating max contribution and min contribution
- Mishandling duplicates in variants
- Forgetting long long

23. Full Problem Walkthrough 12 — LeetCode 42: Trapping Rain Water, Stack Version

23.1 Problem Summary

Given bar heights, compute trapped rainwater.

Example:

height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output:

6

23.2 How to Identify Stack

Water is trapped when a right boundary appears that is taller than a previous valley.

Stack stores decreasing heights.

When current height is greater than stack top, it may form water.

23.3 C++ Code

```
class Solution {
public:
    int trap(vector<int>& height) {
        vector<int> st;
        int water = 0;

        for (int i = 0; i < height.size(); i++) {
            while (!st.empty() && height[i] > height[st.back()]) {
                int bottom = st.back();
                st.pop_back();

                if (st.empty()) {
                    break;
                }

                int left = st.back();
                int width = i - left - 1;
                int boundedHeight = min(height[left], height[i]) - height[bottom];

                water += width * boundedHeight;
            }

            st.push_back(i);
        }

        return water;
    }
};
```

23.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

23.5 Comparison With Two Pointers

Two-pointer solution is $O(1)$ space and often preferred.

Stack solution is useful because it reinforces boundary logic.

24. Full Problem Walkthrough 13 — LeetCode 456: 132 Pattern

24.1 Problem Summary

Find whether there exists a subsequence:

$$\text{nums}[i] < \text{nums}[k] < \text{nums}[j]$$

where:

$$i < j < k$$

This is called a 132 pattern.

Example:

`nums = [3,1,4,2]`

Output:

`true`

Because:

`1, 4, 2`

24.2 How to Identify Monotonic Stack

We need a middle-large value and a later smaller-but-still-bigger-than-left value.
Scan from right to left.

Maintain possible `nums[j]` values in decreasing stack.

Track best `nums[k]`.

24.3 C++ Code

```
class Solution {
public:
    bool find132pattern(vector<int>& nums) {
        int third = INT_MIN;
        vector<int> st;

        for (int i = nums.size() - 1; i >= 0; i--) {
            if (nums[i] < third) {
                return true;
            }

            while (!st.empty() && nums[i] > st.back()) {
                third = st.back();
                st.pop_back();
            }
        }
    }
};
```

```

        st.push_back(nums[i]);
    }

    return false;
}
};

```

24.4 Intuition

third represents the best candidate for `nums[k]`.

If we later find a smaller `nums[i]`, then:

```
nums[i] < third < nums[j]
```

So pattern exists.

24.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

25. Mini Problem 1 — LeetCode 496 With Reverse Scan

Alternative next greater approach:

```

unordered_map<int, int> next;
vector<int> st;

for (int i = nums2.size() - 1; i >= 0; i--) {
    int x = nums2[i];

    while (!st.empty() && st.back() <= x) {
        st.pop_back();
    }

    next[x] = st.empty() ? -1 : st.back();

    st.push_back(x);
}

```

This finds next greater by removing all smaller/equal candidates to the right.

26. Mini Problem 2 — Previous Smaller Element

```
vector<int> previousSmaller(vector<int>& nums) {
    vector<int> answer(nums.size(), -1);
    vector<int> st;

    for (int i = 0; i < nums.size(); i++) {
        while (!st.empty() && nums[st.back()] >= nums[i]) {
            st.pop_back();
        }

        if (!st.empty()) {
            answer[i] = nums[st.back()];
        }

        st.push_back(i);
    }

    return answer;
}
```

Used in histogram and boundary problems.

27. Mini Problem 3 — LeetCode 1944: Number of Visible People in a Queue

Pattern

Monotonic decreasing stack from right.

```
class Solution {
public:
    vector<int> canSeePersonsCount(vector<int>& heights) {
        int n = heights.size();

        vector<int> answer(n, 0);
        vector<int> st;

        for (int i = n - 1; i >= 0; i--) {
            while (!st.empty() && heights[i] > st.back()) {

```

```

        answer[i]++;
        st.pop_back();
    }

    if (!st.empty()) {
        answer[i]++;
    }

    st.push_back(heights[i]);
}

return answer;
}
};

```

Key idea:

A taller/equal person blocks visibility after being seen.

28. Mini Problem 4 — LeetCode 853: Car Fleet

Pattern

Sort by position descending and track fleet arrival times.

```

class Solution {
public:
    int carFleet(int target, vector<int>& position, vector<int>& speed) {
        vector<pair<int, double>> cars;

        for (int i = 0; i < position.size(); i++) {
            double time = (double)(target - position[i]) / speed[i];
            cars.push_back({position[i], time});
        }

        sort(cars.rbegin(), cars.rend());

        int fleets = 0;
        double slowestTime = 0.0;

        for (auto [pos, time] : cars) {
            if (time > slowestTime) {
                fleets++;
                slowestTime = time;
            }
        }

        return fleets;
    }
};

```

```

    }
}

return fleets;
}
};

```

This is not a direct stack implementation, but the logic is monotonic:

- cars behind with smaller/equal arrival time merge into fleet ahead
- only increasing arrival times create new fleets

29. Common Edge Cases for Monotonic Stack

29.1 Empty Array

Return empty answer or 0 depending on problem.

29.2 One Element

Next greater/smaller is usually -1 or 0.

29.3 Strict vs Non-Strict Comparisons

This is very important.

Use carefully:

```

>
>=
<
<=

```

Examples:

- Daily Temperatures needs strictly greater.
- Final Prices needs smaller or equal.
- Sum of Subarray Minimums needs asymmetric duplicate handling.

29.4 Duplicates

Duplicates often decide whether to pop on:

```

>=

```

or:

```

>

```

Wrong duplicate handling causes double counting or missing boundaries.

29.5 Circular Arrays

Need loop $2 * n$ times and use modulo.

29.6 Sentinel Values

Histogram problems often need sentinel 0 to flush the stack.

29.7 Long Long

Contribution problems can overflow int.

Use:

```
long long
```

30. Common Mistakes and Wrong Code

Mistake 1 — Storing Values When Indices Are Needed

Daily Temperatures needs distance:

```
i - prevIndex
```

So store indices.

Mistake 2 — Wrong Stack Direction

For next greater, use decreasing stack.

For next smaller, use increasing stack.

Mistake 3 — Forgetting Unresolved Values

For next greater, unresolved indices remain -1.

Initialize answer:

```
vector<int> answer(n, -1);
```

Mistake 4 — Wrong Width in Histogram

Correct:

```
width = rightBoundary - leftBoundary - 1;
```

Mistake 5 — Ignoring Duplicates in Contribution Problems

For Sum of Subarray Minimums, one side must handle equality differently.

Mistake 6 — Thinking Nested While Means $O(n^2)$

Even though there is a while inside a for loop, each element is popped once.

So total time is $O(n)$.

31. Interview Explanation Templates

31.1 Next Greater Element

I maintain a decreasing stack of unresolved indices. When the current value is greater than the value at the stack top, the current value is the next greater element for that index. I pop and fill the answer. Each index is pushed and popped at most once, so the solution is $O(n)$.

31.2 Daily Temperatures

I store indices of days whose warmer day has not been found yet. When a warmer temperature appears, it resolves all previous colder days at the top of the stack.

31.3 Histogram

I maintain an increasing stack of bar indices. When I see a smaller height, it means the popped bar can no longer extend to the right. Its left boundary is the new stack top, and its right boundary is the current index.

31.4 Sum of Subarray Minimums

Instead of enumerating subarrays, I compute each element's contribution as the minimum. I find how many subarrays can extend left and right while keeping that element as minimum, using monotonic stacks.

31.5 Remove K Digits

To make the number smallest, whenever the current digit is smaller than the previous digit, I remove the previous digit if I still can. This keeps the stack increasing and greedily improves the most significant digits.

32. Complexity Patterns

Next Greater Element	$O(n)$	$O(n)$
Circular Next Greater	$O(n)$	$O(n)$
Daily Temperatures	$O(n)$	$O(n)$
Stock Span	amortized $O(1)$ per call	$O(n)$
Final Prices	$O(n)$	$O(n)$
Remove K Digits	$O(n)$	$O(n)$
Remove Duplicate Letters	$O(n)$	$O(1)$
Largest Rectangle	$O(n)$	$O(n)$
Maximal Rectangle	$O(R * C)$	$O(C)$
Sum of Subarray Minimums	$O(n)$	$O(n)$
Sum of Subarray Ranges	$O(n)$	$O(n)$
Trapping Rain Water Stack	$O(n)$	$O(n)$
132 Pattern	$O(n)$	$O(n)$

33. Time-Boxed Practice Guidance

Intro Monotonic Stack

Target:

15–25 minutes

Problems:

- Next Greater Element I
- Daily Temperatures
- Final Prices With Discount

Core Medium Problems

Target:

25–45 minutes

Problems:

- Next Greater Element II
- Online Stock Span
- Remove K Digits
- Asteroid Collision
- Car Fleet

Hard Monotonic Stack Problems

Target:

45–75 minutes

Problems:

- Largest Rectangle in Histogram
- Maximal Rectangle
- Sum of Subarray Minimums
- Sum of Subarray Ranges
- 132 Pattern
- Number of Visible People in a Queue

34. Expanded Practice Set

34.1 Must-Do Easy

29. Next Greater Element I
30. Final Prices With a Special Discount
31. Daily Temperatures
32. Make The String Great
33. Remove All Adjacent Duplicates in String

34.2 Must-Do Medium

34. Next Greater Element II
35. Online Stock Span
36. Remove K Digits
37. Remove Duplicate Letters
38. Asteroid Collision
39. Car Fleet
40. Number of Visible People in a Queue
41. Buildings With an Ocean View
42. Sum of Subarray Minimums

34.3 Strong Interview Practice

43. Largest Rectangle in Histogram

- 44. Maximal Rectangle
- 45. Sum of Subarray Ranges
- 46. Trapping Rain Water, stack version
- 47. 132 Pattern
- 48. Smallest Subsequence of Distinct Characters
- 49. Most Competitive Subsequence
- 50. Maximum Width Ramp

34.4 Hard / Stretch

- 51. Maximal Rectangle
 - 52. Sum of Subarray Minimums
 - 53. Sum of Subarray Ranges
 - 54. Count Submatrices With All Ones
 - 55. Create Maximum Number
 - 56. Remove K Digits
 - 57. Number of Visible People in a Queue
 - 58. Largest Rectangle in Histogram
-

35. Suggested Study Order

Step 1 — Next Greater Basics

Start with:

- 59. Next Greater Element I
- 60. Daily Temperatures
- 61. Next Greater Element II

Goal:

Understand unresolved decreasing stack.

Step 2 — Next Smaller Basics

Solve:

- 62. Final Prices With a Special Discount
- 63. Previous Smaller Element practice
- 64. Remove K Digits

Goal:

Understand increasing stack and greedy popping.

Step 3 — Design / Online Stack

Solve:

- 65. Online Stock Span

Goal:

Understand compressed stack pairs.

Step 4 — Histogram Boundary Problems

Solve:

- 66. Largest Rectangle in Histogram
- 67. Maximal Rectangle

Goal:

Master previous/next smaller boundaries.

Step 5 — Contribution Problems

Solve:

- 68. Sum of Subarray Minimums
- 69. Sum of Subarray Ranges

Goal:

Learn element contribution counting.

Step 6 — Advanced Monotonic Stack

Solve:

- 70. 132 Pattern
- 71. Number of Visible People in a Queue
- 72. Trapping Rain Water, stack version

Goal:

Handle non-obvious monotonic stack applications.

36. Pattern Decision Table

next greater to right	decreasing stack	Next Greater Element
warmer future day	decreasing stack of indices	Daily Temperatures
circular next greater	decreasing stack + two passes	Next Greater II
next smaller/equal	increasing stack	Final Prices
previous greater	decreasing stack	Stock Span
previous smaller	increasing stack	Histogram
largest rectangle	increasing index stack	LC 84
matrix rectangle	histogram per row	LC 85
remove digits	increasing string stack	Remove K Digits
smallest unique letters	increasing stack + last index	Remove Duplicate Letters
sum of minimums	prev/next smaller boundaries	LC 907
subarray range sum	max contribution - min contribution	LC 2104
visible people	decreasing stack	LC 1944
132 pattern	reverse scan stack	LC 456

37. Final Cheat Sheet

Next Greater

```
vector<int> answer(n, -1);
vector<int> st;

for (int i = 0; i < n; i++) {
    while (!st.empty() && nums[i] > nums[st.back()]) {
        answer[st.back()] = nums[i];
        st.pop_back();
    }

    st.push_back(i);
}
```

Daily Temperatures

```
while (!st.empty() && temperatures[i] > temperatures[st.back()]) {
    int prev = st.back();
    st.pop_back();

    answer[prev] = i - prev;
}
```

Next Greater Circular

```
for (int i = 0; i < 2 * n; i++) {
    int index = i % n;

    while (!st.empty() && nums[index] > nums[st.back()]) {
        answer[st.back()] = nums[index];
        st.pop_back();
    }

    if (i < n) {
        st.push_back(index);
    }
}
```

Histogram

```
heights.push_back(0);

for (int i = 0; i < heights.size(); i++) {
```

```

while (!st.empty() && heights[i] < heights[st.back()]) {
    int h = heights[st.back()];
    st.pop_back();

    int left = st.empty() ? -1 : st.back();
    int width = i - left - 1;

    best = max(best, h * width);
}

st.push_back(i);
}

```

Remove K Digits

```

while (!st.empty() && k > 0 && st.back() > c) {
    st.pop_back();
    k--;
}

st.push_back(c);

```

Contribution Formula

```

contribution = value * choicesOnLeft * choicesOnRight;

```

38. What to Master Before Chapter 15

Before moving to Monotonic Queue / Deque, make sure you can:

- Explain what monotonic stack means
- Know increasing vs decreasing stack
- Solve Next Greater Element
- Solve Daily Temperatures
- Solve Next Greater Element II
- Solve Final Prices
- Solve Online Stock Span
- Solve Remove K Digits
- Solve Remove Duplicate Letters
- Solve Largest Rectangle in Histogram
- Understand histogram width formula
- Understand duplicate handling in contribution problems
- Solve Sum of Subarray Minimums
- Explain why monotonic stack is $O(n)$
- Know when to store indices instead of values

39. Core Takeaway

Monotonic Stack is a stack that keeps useful order while storing unresolved elements.

The most important idea is:

Each element waits in the stack until a future element resolves it.

For next greater:

current greater element resolves previous smaller elements

For next smaller:

current smaller element resolves previous larger elements

For histogram and contribution problems:

monotonic stack finds the nearest boundary where an element stops being useful

The reason monotonic stack is powerful is that it turns repeated scanning into one pass.

Each element is:

- pushed once
- popped at most once

So the total time is $O(n)$, even with a nested while loop.

Monotonic Stack is one of the biggest jumps from basic stack problems to advanced interview patterns.
